Roshan Aditya Murali

Corvallis, OR · +1 458-272-8184 · roshanaditya23@gmail.com · LinkedIn · GitHub

Summary

Software engineer with a year of full-time, full-lifecycle experience shipping cloud-native Java and Node.js services, React/TypeScript front ends, and CI/CD pipelines on AWS, now finishing an M.Eng. in CS with research in program analysis, NLP, and large-scale LLM evaluation. Works daily with AI productivity tooling—Claude Code, LLM tool-use APIs—and built an autonomous coding agent that completes multi-file tasks end to end.

Education

Oregon State University **Sep 2024 – Dec 2026**
M.Eng., Computer Science *Corvallis, OR*
- Coursework: Algorithms & Data Structures, Program Analysis & Evaluation, Software Engineering Methods & Tools, Computer Architecture.

Vellore Institute of Technology **Jul 2019 – Aug 2023**
B.Tech., Computer Science GPA: 3.2/4.0 *India*

Technical Skills

- **Languages:** Python, Java, TypeScript, JavaScript, SQL, C++ (coursework), Bash
- **AI Productivity & Agents:** Claude Code, LLM tool-use APIs (Anthropic, OpenAI), autonomous coding agents, GenAI workflow integration, evaluation harnesses
- **Web & Services:** React, Next.js, Node.js, Spring Boot, REST/OpenAPI, HTML, CSS, Drupal; web accessibility (semantic markup, alt text, screen-reader-friendly tables)
- **Systems & Cloud:** Unix/Linux, Docker, Kubernetes, GitHub Actions CI/CD, distributed microservices, AWS (EC2, RDS, Lambda, DynamoDB, S3, CDK, API Gateway)
- **Data, ML & NLP:** PostgreSQL, MySQL, Hugging Face (BERT, RoBERTa, MiniLM), spaCy, NLTK/VADER, scikit-learn, OpenCV, CNNs, Pandas

Experience

Soreal Inc. **Jun 2023 – Jun 2024**
Software Engineer I
- Cut data latency 30% across customer-facing services by re-partitioning service boundaries, adding caching layers, and hardening failure paths in fault-tolerant Spring Boot and Node.js microservices on AWS (EC2, RDS).
- Delivered real-time customer dashboards in React, Next.js, and TypeScript, owning each feature end to end from requirements and API contract through UI, code review, testing, and production release.
- Enabled integration across distributed backend services without per-consumer glue code by designing and documenting REST APIs to OpenAPI standards using established OOP design patterns.
- Achieved zero-downtime deployments by building GitHub Actions and Docker CI/CD pipelines gated on automated tests, with least-privilege IAM roles, encrypted secrets, and authenticated API access.

ASOSU – Oregon State University **Apr 2026 – Present**
Website Coordinator *Corvallis, OR*
- Own development and the in-progress redesign of the student government website (Drupal 10, Bootstrap, custom HTML/CSS), the public record for OSU legislation, minutes, and student fees.
- Made published records usable with screen readers by rebuilding meeting archives as semantic, accessible HTML tables and transcribing image- and PDF-only documents into structured, alt-texted text.

Ford Motor Company **May 2022 – Aug 2022**
Data Analytics & Front-End Intern
- Reduced manual reporting effort 20% by writing SQL queries and building Power BI KPI dashboards on optimized data pipelines, translating cross-functional requirements into shipped reporting.

Projects

LEST: Specification-Based Safety Testing of LLMs **2026**
Python, spaCy, Hugging Face, LLM APIs (under review, ICSE '27)
- Co-developed a framework that treats LLM system-card safety claims as testable specifications; found only 29 of 56 documented claims are black-box testable and only 17 are covered by an existing benchmark.
- Built a mutation fuzzer with 7 operators and 4 validity filters (grammaticality, perplexity, semantic similarity, lexical distance), generating 1,836 test prompts from 97 SORRY-Bench seeds.
- Evaluated Claude Opus 4.8 and GPT-5.5: human review confirmed only 19 of 312 judge-flagged failures (6.1%), yet these contradicted 6 of 9 covered refusal claims despite 95%+ baseline refusal.

LLM-Inferred Invariants for Patch Correctness Assessment **2026**
Python, Java, Daikon, SMT, Defects4J, Unix/Linux (targeting FSE '26)
- Removed the dependency on ground-truth patches and costly dynamic analysis in automated program repair by inferring program invariants directly from buggy Java source with LLMs.
- Reached accuracy competitive with state-of-the-art patch-correctness methods by combining LLM-inferred invariants, syntactic similarity features, and SMT-based consistency checking, validated against Daikon-generated invariants via shell-scripted Defects4J harnesses on Linux.

Autonomous AI Coding Agent **2026** *Python, LLM APIs, tool use*
- Completed complex multi-file tasks with minimal human intervention via an iterative multi-step tool-calling loop over file read/write, directory search, and Python execution; contributed the framework to an open-source repository.